COMPUTER PROGRAMMING I

(JAVA)

1.1 What is Computer Programming?

Computer programming is the process of giving a computer a set of instructions that tells it **what to do and how to do it**.

For example, suppose we want a computer to calculate the average of three examination scores.

A human being may think:

1. Get the three scores.
2. Add them together.
3. Divide the result by 3.
4. Display the answer.

A computer cannot simply understand these instructions in ordinary English. They must be converted into a programming language.

1.2 What is a Programming Language?

A programming language is a formal language used to communicate instructions to a computer.

Examples include:

- Java
- Python
- C
- C++
- JavaScript
- C#
- Kotlin
- PHP

Different programming languages have different syntax and features, but they generally allow programmers to:

- store information
- perform calculations
- make decisions
- repeat operations

- process data
- communicate with users
- interact with files and databases.

1.3 Why Java?

Java is a general purpose, object oriented programming language widely used for developing different types of software.

Java is useful for students because it introduces important programming concepts that are applicable to many other languages.

Important characteristics of Java

Java is:

- object oriented
- strongly typed
- platform independent through the Java Virtual Machine
- relatively portable
- widely used
- supported by a large standard library.

The common Java principle is:

Write Once, Run Anywhere

This means that compiled Java programs can run on systems that have a compatible Java Virtual Machine.

1.4 JDK, JRE and JVM

Students often confuse these three terms.

JVM

Java Virtual Machine

The JVM executes Java bytecode.

JRE

Java Runtime Environment

It provides the environment required to run Java applications.

JDK

Java Development Kit

It provides tools required to develop Java programs, including the compiler.

The relationship can be simplified as:

```
JDK
 |
 +---- Development Tools
 |
 +---- JRE
        |
        +---- JVM
```

LECTURE 2

PROGRAMMING PARADIGMS

A programming paradigm is a particular way of thinking about and organising a program.

The course introduces four important paradigms.

2.1 Functional Programming

Functional programming treats computation largely in terms of functions.

A function receives input and produces an output.

2.2 Declarative Programming

Declarative programming focuses more on **what should be achieved** than on specifying every step of how to achieve it.

2.3 Logic Programming

Logic programming represents problems using facts and rules.

Languages such as Prolog are associated with this paradigm.

2.4 Object Oriented Programming

Object oriented programming, or OOP, organises software around **objects and classes**.

Java is primarily an object-oriented language.

Important OOP concepts include:

- class
- object
- abstraction
- encapsulation
- inheritance
- polymorphism.

2.5 Scripting Languages

A scripting language is commonly used to automate tasks or control applications.

Examples include:

- JavaScript
- Python
- PHP
- PowerShell
- Bash.

Java can also be used for scripting like tasks, although Java is traditionally regarded as a general purpose compiled language.

2.6 Structured Programming

Structured programming encourages programs to be organised using clear control structures.

The three fundamental structures are:

Sequence

Instructions execute one after another.

```
Read score
Calculate average
Display average
```

Selection

The program makes a decision.

```
If score >= 50
    Display "Pass"
Else
```

```
    Display "Fail"
```

Iteration

The program repeats instructions.

```
Repeat 10 times
    Display number
```

LECTURE 3

VARIABLES AND DATA TYPES

3.1 What is a Variable?

A variable is a named memory location used to store a value.

Example:

```
int age = 20;
```

Here:

- `int` is the data type
- `age` is the variable
- `20` is the value.

The value can change:

```
age = 21;
```

3.2 Java Data Types

Java data types can broadly be divided into primitive and reference types.

Primitive Types

Type	Example
byte	100
short	2000
int	50
long	50000L
float	12.5f
double	12.5

Type	Example
char	'A'
boolean	true

Example:

```
int age = 20;
double salary = 250000.50;
char grade = 'A';
boolean registered = true;
```

3.3 Constants

A constant is a value that should not be changed after it has been assigned.

Java uses `final`.

```
final double PI = 3.14159;
```

Trying to change it later is not permitted.

3.4 Identifiers

An identifier is the name given to a programming element such as:

- variable
- class
- method
- object.

Examples:

```
studentName
totalScore
calculateAverage
Student
```

Good names should clearly describe their purpose.

Compare:

```
int x;
```

with:

```
int studentAge;
```

The second is easier to understand.

3.5 Variable Naming Rules

A Java identifier:

- cannot contain spaces
- cannot begin with a number
- cannot be a Java keyword
- is case sensitive.

For example:

```
studentName
StudentName
```

are different identifiers.

LECTURE 4

EXPRESSIONS, OPERATORS AND ASSIGNMENT

4.1 Assignment

Assignment stores a value in a variable.

```
int age = 20;
```

Later:

```
age = 25;
```

The right hand side is evaluated first, then assigned to the variable.

4.2 Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction

Operator	Meaning
*	Multiplication
/	Division
%	Remainder

Example:

```
int a = 10;
int b = 3;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
System.out.println(a % b);
```

Output includes:

```
13
7
30
3
1
```

Notice that integer division gives an integer result.

4.3 Relational Operators

These compare values.

```
>     greater than
<     less than
>=    greater than or equal
<=    less than or equal
==    equal to
!=    not equal to
```

Example:

```
int age = 20;

System.out.println(age >= 18);
```

Output:

```
true
```

4.4 Logical Operators

Operator	Meaning
`&&`	AND
`` ` ``	
`!`	NOT

Example:

```
int age = 20;
boolean student = true;

if (age >= 18 && student) {
    System.out.println("Eligible");
}
```

4.5 Increment and Decrement

```
x++;
```

means:

```
x = x + 1;
```

Similarly:

```
x--;
```

means:

```
x = x - 1;
```

LECTURE 5

SIMPLE INPUT AND OUTPUT

5.1 Output

The simplest output statement is:

```
System.out.println("Welcome");
```

`print()` does not automatically move to the next line.

```
System.out.print("Hello ");
System.out.print("Students");
```

Output:

```
Hello Students
```

5.2 Input Using Scanner

Java provides `Scanner` for simple keyboard input.

```
import java.util.Scanner;

public class StudentInput {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = input.nextLine();

        System.out.print("Enter your age: ");
        int age = input.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

LECTURE 6

CONTROL STRUCTURES

Control structures determine the order in which statements execute.

6.1 The `if` Statement

```
int score = 65;

if (score >= 50) {
    System.out.println("Pass");
}
```

6.2 `if...else`

```
if (score >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

6.3 Multiple Conditions

```
if (score >= 70) {
    System.out.println("A");
} else if (score >= 60) {
    System.out.println("B");
} else if (score >= 50) {
    System.out.println("C");
} else {
    System.out.println("F");
}
```

6.4 Switch

A `switch` statement is useful when selecting among several discrete choices.

```
int choice = 2;

switch (choice) {
    case 1:
        System.out.println("Add");
        break;

    case 2:
        System.out.println("Edit");
        break;

    case 3:
        System.out.println("Delete");
        break;

    default:
        System.out.println("Invalid choice");
}
```

6.5 The `for` Loop

Used when the number of repetitions is known.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Output:

```
1
2
3
4
```

```
5
```

6.6 The `while` Loop

Used when repetition depends on a condition.

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

6.7 The `do...while` Loop

This executes at least once.

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

LECTURE 7

ARRAYS

An array stores multiple values of the same data type under one variable name.

Instead of:

```
int score1 = 70;
int score2 = 65;
int score3 = 80;
```

we can write:

```
int[] scores = {70, 65, 80};
```

7.1 Accessing Array Elements

Java arrays use zero based indexing.

```
System.out.println(scores[0]);
```

produces:

```
70
```

The indexes are:

```
Index:    0    1    2
Value:   70   65   80
```

7.2 Processing an Array

```
int[] scores = {70, 65, 80, 75, 90};

int total = 0;

for (int score : scores) {
    total += score;
}

double average = (double) total / scores.length;

System.out.println("Average = " + average);
```

This is an important example because it combines:

- arrays
- loops
- variables
- arithmetic
- type conversion
- output.

7.3 Multidimensional Arrays

Example:

```
int[][] marks = {
    {70, 80, 90},
    {60, 75, 85}
};
```

This can represent two students with three scores each.

LECTURE 8

STRINGS AND STRING PROCESSING

A string is a sequence of characters.

Example:

```
String name = "John";
```

LECTURE 9

METHODS AND PARAMETER PASSING

A method is a named block of code that performs a particular task.

Example:

```
static int add(int a, int b) {
    return a + b;
}
```

Calling it:

```
int result = add(10, 20);
```

9.1 Why Use Methods?

Methods help us:

- avoid repetition
- organise programs
- improve readability
- simplify testing
- divide large problems into smaller problems.

9.2 Method With No Return Value

```
static void greet() {
    System.out.println("Welcome to Java");
}
```

9.3 Method With Parameters

```
static void greetStudent(String name) {
    System.out.println("Welcome, " + name);
```

```
}
```

Calling:

```
greetStudent("Mary");
```

9.4 Method Returning a Value

```
static double calculateAverage(double a, double b) {
    return (a + b) / 2;
}
```

9.5 Parameter Passing

Example:

```
static int square(int number) {
    return number * number;
}
```

Calling:

```
square(5);
```

The value `5` is passed to `number`.

Students should understand that Java uses **pass by value**. For object references, the value passed is the reference itself, which often causes confusion and should be demonstrated carefully in class.

8.1 Common String Operations

```
String name = "Computer Science";

System.out.println(name.length());
System.out.println(name.toUpperCase());
System.out.println(name.toLowerCase());
System.out.println(name.charAt(0));
```

8.2 Comparing Strings

Do not normally compare string contents using:

```
name1 == name2
```

Instead:

```
name1.equals(name2)
```

Example:

```
String password = "java123";

if (password.equals("java123")) {
    System.out.println("Correct");
}
```

8.3 String Concatenation

```
String firstName = "John";
String lastName = "Okon";

String fullName = firstName + " " + lastName;

System.out.println(fullName);
```

Output:

```
John Okon
```

8.4 Practical Exercise

Write a Java program that:

1. accepts a student's full name
2. displays the number of characters
3. converts the name to uppercase
4. displays the first character.

5.22. TYPES OF PROGRAMMING ERRORS

6. Syntax Error

7. Occurs when the program violates Java syntax.
8. Example:

```
9. System.out.println("Hello"
```

10. The closing parenthesis is missing.
11.

12. Runtime Error

13. Occurs while the program is executing.
14. Example:
```
15.  int x = 10 / 0;
```
16. ---
17. Logic Error
18. The program runs but produces the wrong result.
19. Example:
```
20.  double average = total / 4;
```
21. when there are actually five scores.
22. The program may execute normally but produce an incorrect answer.

DEBUGGING

Debugging is the process of locating and correcting errors.

Students should learn to:

1. read compiler messages
2. identify the line where the problem occurred
3. inspect variable values
4. trace program execution
5. test boundary cases
6. correct the error

test again.

LECTURE 10

OBJECT ORIENTED PROGRAMMING

Object oriented programming is central to Java.

The major concepts are:

1. Classes
2. Objects
3. Abstraction
4. Encapsulation
5. Inheritance
6. Polymorphism.

10.1 What is a Class?

A class is a blueprint or template for creating objects.

For example:

```
class Student {
    String name;
    int age;
}
```

This describes what a student object may contain.

10.2 What is an Object?

An object is an instance of a class.

```
Student student1 = new Student();
```

We can then assign values:

```
student1.name = "John";
student1.age = 20;
```

10.3 Adding Methods to a Class

```
class Student {

    String name;
    int age;

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

Use:

```
Student student1 = new Student();

student1.name = "John";
student1.age = 20;

student1.displayDetails();
```

10.4 Encapsulation

Encapsulation means keeping an object's data and methods together while controlling how its internal data is accessed.

Example:

```
class Student {

    private String name;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

The field is private.

The methods control access to it.

10.5 Abstraction

Abstraction means focusing on the important features of an object while hiding unnecessary implementation details.

For example, when using an ATM, a user sees:

```
Insert card
Enter PIN
Select transaction
Receive cash
```

The user does not need to know the internal software and hardware operations that process the transaction.

LECTURE 11

PACKAGES AND JAVA APIs

11.1 What is a Package?

A package is a mechanism for organising related Java classes and interfaces.

Example:

```
package university;
```

Packages help with:

- organisation
- naming
- reuse
- access control.

11.2 Importing Classes

Example:

```
import java.util.Scanner;
```

We can then use:

```
Scanner input = new Scanner(System.in);
```

11.3 What is an API?

API means **Application Programming Interface**.

Java provides a large collection of ready made classes and methods.

Instead of writing everything from scratch, programmers can use existing Java libraries.

Examples include:

```
java.util
java.io
java.lang
java.time
```

11.4 List

A `List` stores an ordered collection of elements.

Example:

```
import java.util.ArrayList;
import java.util.List;

List<String> students = new ArrayList<>();

students.add("John");
students.add("Mary");
```

```
students.add("Peter");

System.out.println(students);
```

We can access an element:

```
System.out.println(students.get(0));
```

11.5 Iterator

An iterator allows us to move through elements in a collection.

```
import java.util.*;

List<String> names = new ArrayList<>();

names.add("John");
names.add("Mary");
names.add("Peter");

Iterator<String> iterator = names.iterator();

while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

11.6 Stack

A stack follows:

LIFO: Last In, First Out

Imagine a pile of books.

The last book placed on top is the first book removed.

Conceptually:

```
+-------+
| Book C|  <- removed first
+-------+
| Book B|
+-------+
| Book A|
+-------+
```

Java example:

```
Stack<Integer> stack = new Stack<>();

stack.push(10);
```

```
stack.push(20);
stack.push(30);

System.out.println(stack.pop());
```

Output:

```
30
```

11.7 Queue

A queue follows:

FIFO: First In, First Out

Think about students waiting for registration.

The first student to join the queue is normally the first student served.

```
Queue<String> queue = new LinkedList<>();

queue.add("John");
queue.add("Mary");
queue.add("Peter");

System.out.println(queue.remove());
```

Output:

```
John
```

LECTURE 12

INHERITANCE

Inheritance allows a class to acquire properties and behaviour from another class.

Example:

```
class Person {
    String name;

    void speak() {
        System.out.println("Person is speaking");
    }
}
```

A subclass:

```
class Student extends Person {

    void study() {
        System.out.println("Student is studying");
    }
}
```

Now:

```
Student s = new Student();

s.name = "John";
s.speak();
s.study();
```

The student inherits `name` and `speak()` from `Person`.

12.1 Why Inheritance?

Inheritance can help:

- reduce code duplication
- represent relationships between classes
- promote reuse
- support polymorphism.

LECTURE 13

POLYMORPHISM

Polymorphism means **one interface or reference can be associated with different forms of behaviour**.

Consider:

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
```

Subclass:

```
class Dog extends Animal {

    @Override
```

```
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Another subclass:

```
class Cat extends Animal {

    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
```

Now:

```
Animal a;

a = new Dog();
a.sound();

a = new Cat();
a.sound();
```

The same reference type can produce different behaviour.

This is an important example of **runtime polymorphism**.

LECTURE 14

SEARCHING ALGORITHMS

Searching means finding a particular value in a collection.

Suppose:

```
10  25  30  45  60
```

We want to find `45`.

14.1 Linear Search

Linear search examines elements one after another.

```
static int linearSearch(int[] data, int target) {
```

```
    for (int i = 0; i < data.length; i++) {

        if (data[i] == target) {
            return i;
        }
    }

    return -1;
}
```

If the target is found, the index is returned.

If it is not found:

```
-1
```

14.2 Practical Example

```
int[] numbers = {10, 25, 30, 45, 60};

int result = linearSearch(numbers, 45);

if (result != -1) {
    System.out.println("Found at index " + result);
} else {
    System.out.println("Not found");
}
```

14.3 Binary Search

Binary search is applicable to a sorted collection.

Example:

```
10 20 30 40 50 60 70
```

To find `60`, instead of checking every item, the algorithm repeatedly divides the search area.

Basic idea:

```
Search entire array
       ↓
Check middle
       ↓
Target smaller or larger?
       ↓
Discard irrelevant half
       ↓
Repeat
```

Binary search has logarithmic time complexity, commonly expressed as:

```
O(log n)
```

LECTURE 15

SORTING ALGORITHMS

Sorting means arranging data in a particular order.

Example:

```
Before:
40 10 30 20

After:
10 20 30 40
```

15.1 Bubble Sort

Bubble sort repeatedly compares neighbouring elements.

```
static void bubbleSort(int[] data) {

    for (int i = 0; i < data.length - 1; i++) {

        for (int j = 0; j < data.length - 1 - i; j++) {

            if (data[j] > data[j + 1]) {

                int temp = data[j];
                data[j] = data[j + 1];
                data[j + 1] = temp;
            }
        }
    }
}
```

15.2 Tracing Bubble Sort

Suppose:

```
5 2 4 1
```

First pass:

```
5 2 4 1
2 5 4 1
2 4 5 1
2 4 1 5
```

Largest element has moved toward the end.

Second pass:

```
2 4 1 5
2 1 4 5
```

Continue until sorted:

```
1 2 4 5
```

This type of tracing should be done manually during laboratory sessions.

15.3 Other Sorting Algorithms

Students should also understand the basic ideas behind:

- selection sort
- insertion sort
- merge sort
- quicksort.

At this level, particular attention should be given to implementing and tracing simple sorting algorithms.